

Übungsblatt 7

zur Veranstaltung „RobE – Einführung in die Robotik“

7. Taskgenerierung und Perzeption

In der Übung 7 geht es um die Generierung von Tasks und der Perzeption von Objekten mittels 3D Sensordaten.

Zur Bearbeitung der Übungsaufgaben: Bearbeiten Sie auf jeden Fall *alle* Übungsaufgaben. Ausgenommen hiervon sind lediglich die mit „optional“ gekennzeichneten Textstellen. Lesen Sie sich die Aufgaben gut durch. Sollten Sie eine Aufgabe nicht lösen können, so beschreiben Sie zumindest, wie weit Sie gekommen sind und auf welche Weise Sie vorgegangen sind. Aufgaben mit der Randbemerkung „*Protokoll!*“ sind im Protokoll zu beantworten. Als Lösung der Aufgaben ist (je nach Aufgabe) Programmcode abzugeben (kommentiert) oder ein kurzer Text.

7.1. Vorbereitungsaufgaben

– keine –

7.2. Praktikumsaufgaben

1. Bearbeiten sie die Schritte “Installation of the Task Constructor” (Anhang A) und “Starting the demonstration” (Anhang B).
2. Bearbeiten Sie die Schritte “Modifying a motion task list” (Anhang C). Fügen Sie nach dem zweiten Task einen Weiteren hinzu, bei dem sich der Roboterarm um 0.2 m nach unten bewegt. Wiederholen Sie diesen Schritt mit einer Verschiebung um 0.4 m nach oben. Waren alle *tasks* erfolgreich? *Protokoll!*
3. Welche Struktur haben die Nachrichten *Vector3Stamped* und *TwistStamped* des Typs *geometry_msgs*? *Protokoll!*
4. Bearbeiten Sie die Schritte “Pick-and-place demonstration with the *Task Constructor*” (Anhang D). Wie viele unterschiedliche *Pick*-Variationen wurden berechnet und wie viele davon erfolgreich? Überlegen Sie, ob es sinnvoll ist, mehrere Variationen zu berechnen. *Protokoll!*
5. Was sind die vordefinierten Maße der Objekte *table* und *cylinder*? *Protokoll!*
6. Bearbeiten Sie die Schritte “Using the Perception Pipeline” (Anhang E). Um was für einen Gegenstand handelt es sich bei dem erfassten Objekt? Welche Vorteile und Nachteile hat es, die Auflösung der *3D Occupancy Grid Map* höher zu stellen? *Protokoll!*
7. Was stellt die gefilterte Point Cloud da? Überlegen Sie, was für einen Nutzen diese haben könnte. *Protokoll!*

A. Installation of the task constructor

Start by downloading and installing the necessary packages and building the source folder:

```
1 echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
2 echo "export PATH=/usr/lib/ccache:$PATH" >> ~/.bashrc
3 source ~/.bashrc
4 sudo apt-get update
5 sudo apt-get install ros-noetic-moveit
  ros-noetic-panda-moveit-config ros-noetic-moveit-visual-tools
6 mkdir -p moveit_ws/src
7 cd moveit_ws/src
8 git clone https://github.com/ros-planning/moveit_tutorials
9 git clone https://github.com/ros-planning/moveit_task_constructor
10 rosdep update
11 rosdep install --from-paths . --ignore-src --rosdistro noetic -y
12 catkin build
13 echo "source ~/moveit_ws/devel/setup.bash" >> ~/.bashrc
```

B. Starting the demonstration

This package has three different demonstrations that show how the *task constructor* functions. To start, we need to launch the *RVIZ* environment that includes the *Panda robot arm*. It will start in a predefined initial state.

Launch the *RVIZ* environment:

```
14 source ~/moveit_ws/devel/setup.bash
15 roslaunch moveit_task_constructor_demo demo.launch
```

A new *Motion Planning Tasks*-window can be seen on the right. Though, there aren't any tasks to find yet. Will will change that by adding some.

Run the following demonstration:

```
16 rosrun moveit_task_constructor_demo cartesian
```

Taking a look at the *RVIZ* environment it can be seen, that tasks have now been added.

The *Panda robot arm* will automatically start with execute the tasks one by one. Each of them will be calculated, once they are next. The task manager also discriminated between a successful and a failed inverse kinematics calculation. A "1" indicates, that a solution has been found.

C. Modifying a motion task list

In order to edit the demonstration, we have to take a look at its source code:

```
17 cd ~/moveit_ws/src/moveit_task_constructor/demo/src
18 gedit cartesian.cpp
```

Its code structure shows us that each task is within brace brackets. This creates a control variable scope for declarations and definitions within that brace block, that are destroyed after going out of scope.

We can add a additional relative motion by simply copying a block, inserting it and editing the motion, that is described by direction vector. Naming it correctly will help identifying it in task list.

After saving the file, the *catkin workspace* has to be rebuild:

```
19 catkin build
```

We can now test our modified motion task:

```
20 source ~/moveit_ws/devel/setup.bash
21 rosrn moveit_task_constructor_demo cartesian
```

D. Pick-and-place demonstration with the Task Constructor

The *Movelt! Task Constructor* also has a *Pick-and-Place* demonstration, though it is a bit more complex than our last example.

Start the demonstration:

```
22 source ~/moveit_ws/devel/setup.bash
23 roslaunch moveit_task_constructor_demo demo.launch
24 roslaunch moveit_task_constructor_demo pickplace.launch
```

This demonstration consists of multiple files.

The tasks for the *Pick-and-Place* tasks can be viewed in:

```
25 cd ~/moveit_ws/src/moveit_task_constructor/demo/src
26 gedit pick_place_task.cpp
```

The operation itself is described in:

```
27 gedit pick_place_demo.cpp
```

Pose names of predefined poses, object dimensions and other important parameters are stored in the file *panda_config.yaml*, that can be found here:

```
28 cd ~/moveit_ws/src/moveit_task_constructor/demo/config
29 gedit panda_config.yaml
```

E. Using the Perception Pipeline

The *Perception Pipeline* allows the visualization of obstacles that were captured with 3D sensors using a 3D occupancy grid. *MoveIt!* in this regard is using *Octomap* to create that map.

Before starting, we will increase the resolution of the 3D occupancy grid.

The settings can be found in the *launch*-folder of the *panda_moveit_config*-package:

```
30 cd ~/moveit_ws/src/panda_moveit_config/launch
```

Edit the file and set the resolution to *0.01*:

```
31 gedit sensor_manager.launch.xml
```

Launch the demo:

```
32 rosrn moveit_tutorials detect_and_add_cylinder_collision_object  
_demo
```

We now can see the 3D occupancy map that has detected an obstacle and set an collision around it. Let's verify that by looking at the sensor data.

Go to *Add*, select *By topic* and choose *PointCloud2* of the depth-field camera on the top of the menu. But in order to see the sensor data, we need to disable the occupancy grid. For that, go to *Planning Scene* in the *Displays* window, open *Scene Geometry* and deselect *Show Scene Geometry*.

To visualize the filtered point cloud, repeat the previously described steps and select *PointCloud2* of the topic */filtered_cloud* instead. Again, deselect or remove the first point cloud.

NOTE: If nothing is displayed, just reselect the previous planning scene and point cloud!